CHAPTER 1

Artificial Neural Networks

"I think the brain is essentially a computer and consciousness is like a computer program. It will cease to run when the computer is turned off. Theoretically, it could be re-created on a neural network, but that would be very difficult, as it would require all one's memories."

-Stephen Hawking



Learning Objectives

Understand the structure and function of artificial neural networks
Learn how to design and implement basic neural networks
Explore the applications of neural networks in various domains

1.1 Introduction to Artificial Neural Network

An artificial neural network (ANN) is a computing system inspired by the biological neural networks found in the human brain. At its core, it is a mathematical framework for learning relationships and patterns from data. The fundamental building block of a neural network is the artificial neuron, a simple unit that receives input, performs a small calculation, and produces an output. These neurons are densely interconnected in structured layers, typically starting with an input layer that receives the initial data, followed by one or more hidden layers where the complex computations and pattern recognitions occur, and finally an output layer that delivers the network's final prediction or decision.

The true power of a neural network comes from its ability to learn. During a training process, the network is presented with vast amounts of data, and it makes initial guesses. The errors in these guesses are then measured, and this error information is sent backward through the network in a process called backpropagation. This process adjusts the strength of the connections between the neurons, which are known as weights. By iteratively adjusting these weights across millions of examples, the network gradually tunes itself, strengthening connections that lead to correct answers and weakening those that cause errors. Over time, it discovers intricate, hierarchical patterns within the data, effectively building an internal representation of the knowledge it needs to perform its task, whether that is recognizing a face in a photo, translating languages, or predicting future stock prices.

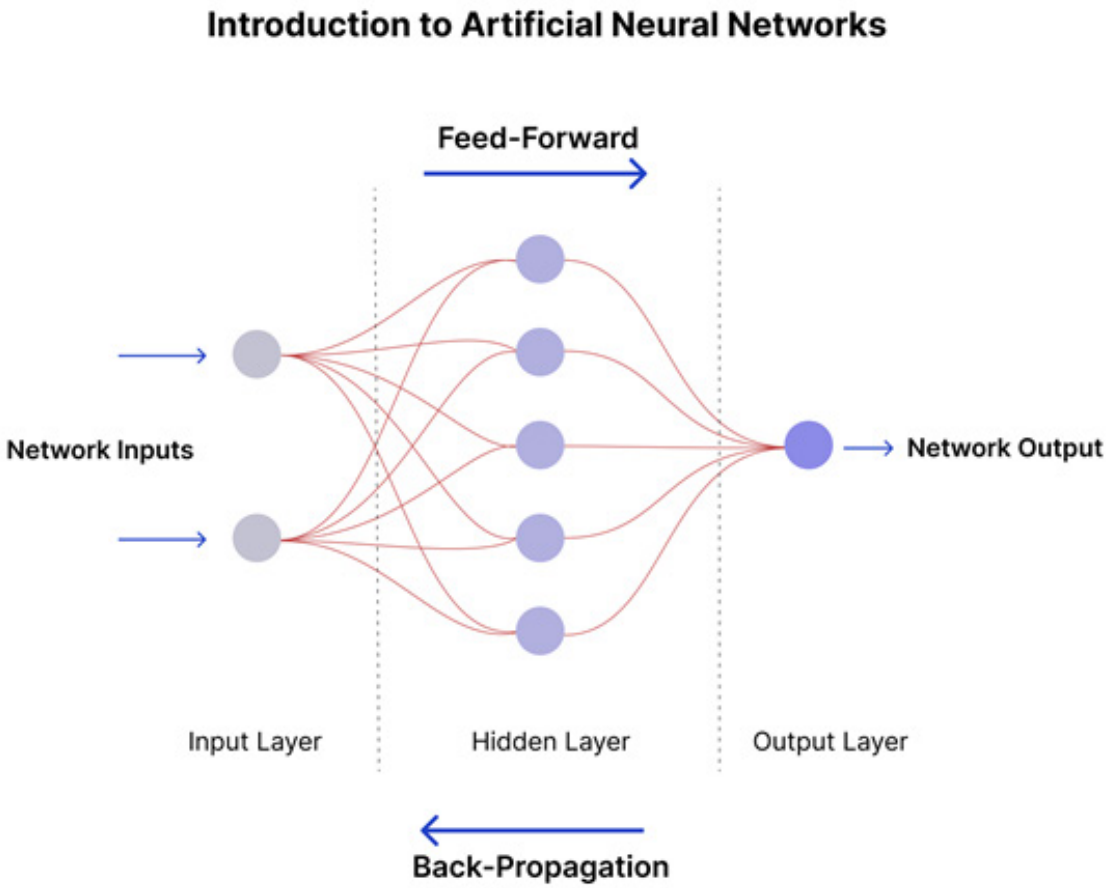


Figure 1.1 Architecture of ANN

1.2 Key Components of ANN

The key components of ANN include input layer, hidden layer and output layer. Imagine an artificial neural network as a sophisticated assembly line for processing information. This assembly line is organized into three distinct stations: the input layer, the hidden layers, and the output layer. The journey of data begins at the input layer, which acts as the receiving dock. Its sole purpose is to accept the raw data from the outside world and distribute it to the rest of the network. Each neuron in this layer corresponds to a single feature of the input data; for instance, in an image, each neuron might represent the brightness of one pixel. This layer performs no calculations itself; it simply serves as the entry point, standardizing and feeding the information forward.

The data then travels to the hidden layers, which are the true heart of the neural network's computational power. These layers are called "hidden" because they are

sandwiched between the input and output, and their internal processes are not directly observable from the outside. This is where the network performs its complex work of identifying patterns, building abstractions, and understanding the intricate relationships within the data. A simple network may have only one hidden layer, while a deep learning network has many. Each successive hidden layer can recognize more complex features; the first might detect simple edges in an image, the next might combine these edges to form shapes, and a later one might assemble those shapes into a recognizable object like a face or a car.

Finally, the processed and transformed information arrives at the output layer, the network's delivery platform. This layer is responsible for translating the abstract computations of the hidden layers into a meaningful and actionable result. The structure of this layer is entirely determined by the task at hand. For a classification task, such as identifying an animal in a photo, each output neuron might represent a different animal class like "cat," "dog," or "horse," and the neuron with the highest activation provides the network's answer. For a regression task, like predicting a house price, there might be just a single neuron that outputs a continuous number. In essence, the input layer receives the question, the hidden layers perform the deep thinking, and the output layer delivers the final answer.

1.3 Working of ANN

The working of an Artificial Neural Network can be thought of as a sophisticated, multi-stage data refinement process inspired by the human brain. It all begins when raw data is presented to the input layer, which acts as the network's sensory interface. Each neuron in this layer receives a piece of the data, such as a pixel from an image or a word from a text. This layer does no calculation; its job is simply to receive and distribute the information to the next layer.

The real magic unfolds in the hidden layers. As data leaves the input layer, it travels along weighted connections. Each weight acts like a volume knob, amplifying or dampening the importance of a particular input. Inside a hidden layer neuron, all these weighted inputs are summed together, and a value called a bias is added to this total, allowing the neuron to fine-tune its trigger point. This combined value is then passed through a crucial gatekeeper known as the activation function. This function, which is often non-linear, decides whether the neuron should "fire" and pass a signal forward. It is this non-linearity that allows the network to learn complex, real-world patterns rather than just simple straight-line relationships. Deep networks have multiple hidden layers, with each successive layer building on the previous one to recognize more abstract and complex features.

This forward journey of data, from input through the hidden layers to the output, is known as forward propagation. At the end of this journey, the output layer produces the network's final prediction. However, the network is not born intelligent; it learns through a critical process called training. During training, the network's initial prediction is compared to the known correct answer, and the difference between them is calculated as an error using a loss function.

This error is then sent backward through the network in a process called backpropagation.

Think of backpropagation as a feedback system that identifies which neurons and connections are most responsible for the mistake. An optimization algorithm, most commonly Gradient Descent, then uses this information to carefully adjust the weights and biases of all the connections. It tweaks them just enough to ensure that, for the same input next time, the network's output will be slightly closer to the correct answer. This entire cycle—forward propagation, error calculation, backpropagation, and weight adjustment—is repeated millions of times with vast amounts of data. With each iteration, the network incrementally improves, steadily refining its internal model of the world. Through this iterative, self-correcting mechanism, the ANN gradually transforms from a collection of random connections into a powerful predictive model, capable of making accurate decisions on never-before-seen data.

1.4 How do Artificial Neural Networks learn?

The learning process of an Artificial Neural Network is a beautifully orchestrated, iterative cycle of guesswork, criticism, and self-improvement. It is not a process of being programmed with rules, but one of discovering patterns through repeated exposure to data.

Imagine teaching a child to identify animals by showing them many pictures. The child makes a guess, you tell them if they are right or wrong, and they adjust their mental model for the next time. An ANN learns in a very similar, though mathematically formalized, way.

The entire process rests on four key stages that repeat in a loop millions of times:

First, the network makes a "Forward Pass" or prediction. The raw input data—let's say an image of a cat—is fed into the input layer. This data then propagates forward through the hidden layers. As it travels, each connection between neurons applies a "weight" (a measure of importance), and each neuron sums its inputs, adds a "bias" (an adjustment factor), and passes the result through an "activation function" to decide whether and how strongly to signal the next layer. This process transforms the raw pixels step-by-step, with each layer recognizing more complex features, until the output layer delivers its final answer. For example, it might output a 70% probability for "cat," a 25% probability for "dog," and a 5% probability for "car."

Second, the network's performance is "Evaluated" using a Loss Function. The network's guess is now compared to the ground truth—the known label that says "this is definitely a cat." A special function called the "Loss Function" (or Cost Function) calculates the error. It quantifies exactly how wrong the network was. A simple loss function might be the square of the difference between the predicted probability and the actual value (1 for cat, 0 for others). The goal of the entire learning process is to minimize this overall error across all the training examples.

Third, the error is sent backward in a process called "Backpropagation." This is the crucial corrective step. The calculated error is propagated backward through the network, from the output layer, through the hidden layers, and all the way back to the input layer. As it travels backward, a powerful calculus technique called the chain rule is used to determine the exact contribution of each weight and bias to the final error. It answers the question: "Which connections were most responsible for this mistake, and

in which direction should they be adjusted to reduce the error?"

Finally, the network "Updates" itself using an Optimizer. Armed with the information from backpropagation, the network now adjusts all of its internal parameters. An optimization algorithm, most commonly a variant of Gradient Descent, performs this update. It looks at the "gradient" (which indicates the direction of steepest ascent for the error) and then tweaks each weight and bias by a small amount in the opposite direction—the direction that will cause the error to decrease. This small amount is controlled by the "Learning Rate," a critical setting that determines the size of the steps taken. A good learning rate allows the network to converge smoothly; a poor one can cause it to overshoot or learn too slowly.

This cycle—forward pass, loss calculation, backpropagation, and weight update—is repeated for every batch of data in the training set, and the entire training set is passed through the network multiple times (in epochs). With each iteration, the network's predictions become slightly more accurate. The weights and biases slowly converge to values that transform the raw input into the correct output, effectively encoding the discovered patterns and relationships directly into the fabric of the network's architecture. The network isn't memorizing; it is building a generalized internal model of the world presented in its training data.

1.5 Artificial neurons vs Biological neurons

Feature	Biological Neuron	Artificial Neuron
Basic Unit	A nerve cell in the brain, with a cell body, dendrites, and an axon.	A mathematical function or computational unit within a software model.
Processing	Electrochemical signals. Electrical impulses (action potentials) and neurotransmitters.	Numerical calculations. Multiplication, addition, and activation functions.
Input	Receives chemical signals from other neurons via dendrites.	Receives numerical data from other neurons or an input source.
Processing Mechanism	Sums incoming signals. If the cumulative excitation exceeds a threshold, it "fires" an action potential.	Sums weighted inputs, adds a bias, and passes the result through an activation function to produce an output.

Output	Transmits an electrical impulse (action potential) along the axon to other neurons.	Outputs a single numerical value, which is sent to other connected artificial neurons.
Speed & Energy	Relatively slow (milliseconds per operation) but highly energy-efficient.	Relatively slow (milliseconds per operation) but highly energy-efficient.
Learning Mechanism	Synaptic plasticity. The strength of connections between neurons changes based on experience (e.g., Hebbian theory: "cells that fire together, wire together").	Adjusting weights and biases through optimization algorithms like Backpropagation and Gradient Descent.
Complexity & Connectivity	Massively parallel, with trillions of connections (synapses). Highly complex, non-linear, and dynamic.	Massively parallel, with trillions of connections (synapses). Highly complex, non-linear, and dynamic.
Primary Function	Information processing, learning, and control of biological systems. Enables cognition, sensation, and motion.	Pattern recognition, function approximation, and data-driven prediction. It is used for tasks like classification and regression.

1.6 Perceptron in ANN

The fundamental building block of modern Artificial Neural Networks (ANNs) is perceptron.

1.6.1. The Core Idea: A Biological Analogy

The architecture of the perceptron is directly inspired by the information processing mechanisms of the biological neuron. A neuron functions by receiving input signals

through its dendrites, which act as the cell's receptive fields. These signals are then aggregated and processed within the soma, or cell body. The soma performs a critical integration function, and if the cumulative input signal surpasses a specific biological threshold, the neuron activates, transmitting an output signal along its axon to subsequent neurons.

A **perceptron** is a simplistic mathematical model of this biological neuron. It's a function that takes several binary inputs, weighs their importance, and produces a single binary output.

1.6.2. The Perceptron Model: The "Recipe"

Think of a perceptron as a simple decision-maker. Let's use the example of **deciding whether to go for a walk.**

Factors (Inputs):

- x_1: Is the weather nice? (Yes=1, No=0)
- x_2: Do I have free time? (Yes=1, No=0)
- x_3: Is my friend coming? (Yes=1, No=0)

Now, not all factors are equally important. The weather might matter more than your friend joining you.

Weights:
Each input has a weight ww, which represents its importance

- w_1=3 (Weather is very important)
- w_2=2 (Free time is important)
- w_3=1 (Friend coming is nice, but less critical)

The Decision Process:

1. Calculate the Weighted Sum: Multiply each input by its weight and add them all up.
   Sum=$(x_1*w_1)+(x_2*w_2)+(x_3*w_3)$

2. Add a Bias: The bias bb is like a threshold. It's how easy or hard it is to get the perceptron to "fire" (output 1). A negative bias means you need stronger evidence to go for a walk.
   Let's say b=−2. This means we need the evidence to be stronger than "2" to make a positive decision.

3. Apply an Activation Function: This function looks at the final sum and makes the yes/no decision.The original perceptron used a Heaviside Step Function.

$$\text{Output} = \begin{cases} 1 & \text{if Sum}+b>0 \\ 0 & \text{if Sum}+b\leq 0 \end{cases}$$

1.6.3. A Concrete Example

Let's run through two scenarios.

Scenario 1: A Great Day for a Walk

- **Inputs:** Nice weather (x_1=1), Free time (x_2=1), Friend can't come (x_3=0)
- **Calculation:**
  - Weighted Sum = (1 * 3) + (1 * 2) + (0 * 1) = 5
  - Add Bias: 5 + (-2) = 3
  - Activation: Is 3 > 0? **Yes!**
- Output: 1➔ **Go for a walk!**

Scenario 2: A Not-So-Great Day

- **Inputs:** Bad weather (x_1=0), Free time (x_2=1), Friend is coming (x_3=1)
- **Calculation:**
  - Weighted Sum = (0 * 3) + (1 * 2) + (1 * 1) = 3
  - Add Bias: 3 + (-2) = 1
  - Activation: Is 1 > 0? Yes! Wait, the output is still 1.
  - `Let's adjust the bias to be more strict. Let's set b=− 4`
  - New Calculation: 3 + (-4) = -1
  - Activation: Is -1 > 0? **No.**
- Output: 0 ➔ **Don't go for a walk.**

The weights and bias are what the perceptron learns during training.

1.6.4. The Perceptron as a Building Block for ANNs

A single perceptron is a **linear binary classifier**. It can only learn to separate data with a straight line (or a hyperplane in higher dimensions). This is a major limitation for complex problems like image recognition, where the patterns are not linearly separable. This is where **Artificial Neural Networks (ANNs)** come in.

How ANNs extend the idea:

1. **Multiple Layers:** An Artificial Neural Network is constructed by connecting many perceptrons together across multiple layers to process information systematically. The architecture begins with an input layer that receives raw data, such as the pixels of an image, and passes this information forward through one or more hidden layers situated between the input and output stages. Within these hidden layers, each individual perceptron specializes in detecting distinct features, learning to recognize elements like edges, curves, or colors from the data it receives. Finally, the output layer synthesizes all this learned information to produce the final result, which might be a classification such as identifying whether an image contains a cat or a dog.

2. **Non-Linear Activation Functions:** Instead of the simple step function, modern ANNs use functions like **Sigmoid, Tanh,** or **ReLU**. These functions allow the network to learn and represent complex, non-linear relationships in the data.
3. **Learning Algorithm:** While a single perceptron uses a simple rule to learn, ANNs use a more powerful algorithm called **Backpropagation** combined with **Gradient Descent** to adjust the thousands or millions of weights and biases efficiently.

Topic of Class Discussion

We see that the core components enabling learning are weights, biases, and activation functions. If you had to prioritize one of these three as the most important for a network's ability to learn complex patterns, which would you choose and why?

1.7 Multilayer Perceptron in ANN

A single perceptron has major limitations - it can only solve linearly separable problems. Real-world problems are rarely that simple. The Multilayer Perceptron solves this by connecting multiple perceptrons in layers, creating a network that can learn complex, non-linear relationships.

1.7.1 MLP Architecture: The Blueprint

A Multilayer Perceptron is built upon a structured arrangement of layers, beginning with an input layer that receives the raw data for processing. Following this are one or more hidden layers, which serve as the core computational engine where the network develops its understanding by learning hierarchical features from the information provided. The architecture culminates in an output layer, which synthesizes everything learned in the preceding layers to generate the final prediction.

Key Components:

A Multilayer Perceptron comprises several key components that work together to enable learning and prediction. At its foundation are stacked layers of perceptrons, with each neuron connected to others through weights and biases that determine the strength and influence of these connections. What allows the network to move beyond simple linear relationships and learn complex patterns is the application of non-linear activation functions within these neurons. The actual processing of information occurs through forward propagation, where data flows systematically from the input layer through the hidden layers and finally to the output. However, the true learning happens through backpropagation, a sophisticated algorithm that iteratively adjusts the weights and biases throughout the network based on the errors it encounters, gradually improving the model's accuracy over time.

1.7.2 Concrete Example: Image Recognition for Handwritten Digits

Let's build an MLP to recognize handwritten digits (0-9) from 28x28 pixel images.

Network Architecture:

MLP Architecture for MNIST digits
Input Layer: 784 neurons (28x28 pixels flattened)
Hidden Layer 1: 128 neurons (with ReLU activation)
Hidden Layer 2: 64 neurons (with ReLU activation)
Output Layer: 10 neurons (with Softmax activation - for digits 0-9)

The Step-by-Step Process:

Step 1: Input Processing

- Each pixel value (0-255) is normalized to 0-1
- The 28x28 grid is flattened into a 784-dimensional vector
- Input: [0.1, 0.8, 0.2, ..., 0.9] (784 values)

Step 2: First Hidden Layer Computation

Each neuron in Hidden Layer 1 calculates:

neuron_output = ReLU((w_1*x_1 + w_2*x_2 + ... + w_{784}*x_{784}) + bias)
Where:

- $w_1 \ldots w_{784}$ are learned weights
- $x_1 \ldots x_{784}$ are input pixel values
- bias is the neuron's bias term
- **ReLU** = max(0, x) - introduces non-linearity

What's happening: Each neuron learns to detect specific features - some might activate for curves, others for straight lines, corners, etc.

Step 3: Second Hidden Layer

The process repeats - now each neuron in Hidden Layer 2 combines outputs from Hidden Layer 1 to detect more complex patterns:

- Neuron A might detect: "curve at top + straight line at bottom" → possibly a '2'
- Neuron B might detect: "closed loop + no vertical lines" → possibly a '0'

Step 4: Output Layer

The final layer uses **Softmax** activation to produce probabilities:

Output = [0.01, 0.02, 0.85, 0.03, 0.01, 0.02, 0.01, 0.02, 0.02, 0.01]
↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑
0 1 2 3 4 5 6 7 8 9
The network predicts "2" with 85% confidence.

1.7.3 Real-World MLP Applications

Application	Input	Hidden Layers Learn	Output
Spam Detection	Email text	Patterns in words and phrases	Spam/Ham probability
Credit Scoring	Financial data	Risk patterns	Credit score
Medical Diagnosis	Patient symptoms & tests	Disease symptom correlations	Diagnosis probability
Voice Recognition	Audio features	Phoneme patterns	Transcribed text

1.7.4. Why MLPs Work So Well

The remarkable effectiveness of Multilayer Perceptrons stems primarily from their capacity for hierarchical feature learning and their theoretical foundation in the Universal Approximation Theorem. Through hierarchical feature learning, these networks progressively build understanding across their layers, with the first hidden layer specializing in detecting simple features such as edges and corners in images or basic word patterns in text. As information moves deeper, the second hidden layer synthesizes these simple features into more complex representations like shapes or phrases, while subsequent layers continue this abstraction process to learn increasingly sophisticated concepts, ultimately recognizing objects or grasping semantic meaning. Complementing this architectural strength is the Universal Approximation Theorem, which provides the mathematical guarantee that even an MLP containing just one hidden layer can approximate any continuous function with sufficient precision, provided it has enough neurons to do so. This theoretical underpinning establishes MLPs as exceptionally powerful and versatile function approximators capable of modeling extraordinarily complex relationships in data.

1.7.5. Comparison between single perceptron and multilayer perceptron

Aspect	Single Perceptron	Multilayer Perceptron (MLP)
Structure	One neuron	Multiple layers of neurons
Capability	Linear decisions	Complex, non-linear decisions
Learning	Simple rules	Backpropagation + Gradient Descent
Use Cases	AND/OR gates	Image recognition, NLP, prediction
Power	Limited	Universal function approximator

The MLP is the workhorse of deep learning - while more advanced architectures like CNNs and Transformers exist for specific domains, the fundamental principles of layered processing, weighted connections, and backpropagation originated with the Multilayer Perceptron.

Example 1.1

The Problem: Streamlining Loan Application Approval

Business Problem: A regional bank wants to improve the efficiency and accuracy of its personal loan approval process. The current method relies heavily on manual review by loan officers, which is slow, can be inconsistent between different officers, and sometimes misses subtle patterns of risk. The bank wants to develop an automated system that can quickly and reliably predict whether an applicant is a good candidate for a loan.

Solution:
The bank decides to build a Multilayer Perceptron (MLP) classifier to predict loan default risk.

Step-by-Step Implementation

1. Data Collection and Preparation

Input Data: The bank gathers historical data from past loan applications. For each applicant, they have features like:

- Annual Income
- Credit Score
- Loan Amount
- Debt-to-Income Ratio (DTI)
- Employment Length
- Number of Existing Loans
- Home Ownership Status (Rent, Own, Mortgage)

Target Output: The most crucial piece of data is the known outcome for these past loans: Loan Status (Fully Repaid vs. Defaulted).

Preprocessing: The data is cleaned (handling missing values) and normalized. For example, Annual Income and Loan Amount are scaled down to a similar range (e.g., 0-1) so that one feature doesn't dominate the others simply because of its larger numerical values. Categorical data like Home Ownership is converted into numerical form.

Designing the Neural Network Architecture
The data science team designs an MLP with the following structure:

- Input Layer: 6 neurons (one for each numerical feature: Income, Credit Score, etc.).
- Hidden Layers:
  - First Hidden Layer: 10 neurons with a ReLU activation function. This layer will start to detect simple, non-linear relationships, like "a high DTI is bad, but a very high income can offset it."
  - Second Hidden Layer: 5 neurons with a ReLU activation function. This layer combines the simple patterns from the first layer to form more complex concepts, such as "applicants with medium income, high credit score, and low existing debt are consistently low-risk."
- Output Layer: 1 neuron with a Sigmoid activation function. This function will output a single probability score between 0 and 1, interpreted as the "probability of default." A score above 0.5 would lead to a "Reject" recommendation, and below 0.5 to an "Approve" recommendation.

3. Training the Model

- The preprocessed historical data is fed into the network.
- Forward Propagation: An applicant's data flows through the network, and based on the initial random weights, it makes a prediction (e.g., a 30% probability of default).
- Loss Calculation: The network's prediction is compared to the actual outcome (e.g., this applicant did not default). The Loss Function (e.g., Binary Cross-

Entropy) calculates how wrong the prediction was.

- Backpropagation & Gradient Descent: The error is sent backward through the network. The Backpropagation algorithm calculates how much each connection weight contributed to the error. The Optimizer (e.g., Adam, a variant of Gradient Descent) then adjusts all the weights
- slightly to reduce the error for the next time it sees similar data.
- This cycle repeats thousands of times over all the historical loan records. With each iteration, the network incrementally improves, learning the complex, non-linear patterns that separate reliable borrowers from high-risk ones.

4. Deployment and Real-World Use

After training is complete and the model's accuracy is validated on a held-out test dataset, it is integrated into the bank's application system.

The New Workflow:

1. A new applicant submits their details online.
2. The data is preprocessed in the same way as the training data.
3. It is fed into the trained ANN.
4. The ANN instantly outputs a probability score (e.g., 0.15 or 15% probability of default).
5. Based on a predefined threshold (e.g., scores below 0.3 are auto-approved, scores above 0.7 are auto-rejected, and scores in between are flagged for manual review), the system provides a recommendation.

Results and Benefits:

- Efficiency: 80% of applications are now processed instantly without human intervention.
- Consistency: The same objective criteria are applied to every single applicant, eliminating human bias and inconsistency.
- Accuracy: The ANN can identify subtle, complex risk patterns that a human officer might miss, leading to a 15% reduction in default rates while also approving more good candidates who were previously overlooked.
- Scalability: The system can easily handle a surge in applications without the bank needing to hire and train more loan officers.

This example demonstrates how an ANN transforms raw data into an intelligent, automated decision-making tool that provides a tangible competitive advantage.

1.8 Introduction to backpropagation

Let's break down Backpropagation in a simple, intuitive way with a detailed example.

1.8.1 The Big Idea: "Learning from Mistakes"

Imagine teaching a child to identify animals. You show them a picture of a cat. If they say "dog," you tell them they're wrong. The child then thinks, "Okay, what in my thought process led me to that wrong answer? I'll adjust my mental model for next time."

Backpropagation is this exact process for an Artificial Neural Network (ANN). It's the core algorithm that allows a network to learn from its errors and adjust its internal parameters (weights and biases) to improve its performance.

The name itself gives a clue:

Propagation: Data flows forward through the network to make a prediction.

Back-Propagation: The error (difference between prediction and actual answer) is sent backward through the network.

The goal is to answer a crucial question: "How much did each neuron contribute to the final error?" Once we know that, we can punish the neurons that caused a big error and reward those that helped.

1.8.2. The Key Ingredient: Gradient Descent & The Chain Rule

Backpropagation is essentially an application of the Chain Rule from calculus to perform Gradient Descent.

Gradient Descent: Imagine you're on a foggy mountain (the error landscape) and want to get to the bottom (minimum error). You look at the slope (gradient) at your feet and take a small step downhill. Backpropagation calculates the slope for every weight in the network.

Chain Rule: Since the final error is a complex function of all the weights in the network (e.g., Error = $f(w_3, g(w_2, h(w_1)))$), the chain rule allows us to calculate the gradient for each weight by multiplying the gradients layer by layer, from the output back to the input.

1.9 Activation Functions

An activation function is a mathematical operation applied to the output of a neuron within a neural network, serving the critical purpose of determining whether and to what degree that neuron should be activated or fired in response to the input it receives. To understand its importance, it is helpful to consider what happens without activation functions, where a neural network would simply perform a series of linear transformations through matrix multiplications, ultimately behaving no differently than a single linear model and therefore possessing the inherent limitation of learning only linear relationships. The introduction of activation functions fundamentally transforms this dynamic by injecting non-linearity into the network, which empowers it to learn the complex, multifaceted patterns characteristic of real-world data and, as formalized by the Universal Approximation Theorem, enables it to approximate virtually any function with sufficient accuracy.

1.9.1. Key Properties of Activation Functions

Activation functions possess several essential properties that collectively enable neural networks to learn effectively. The most fundamental of these is non-linearity, which allows the network to move beyond simple linear relationships and capture the complex patterns inherent in real-world data. Equally important is differentiability, as this property ensures that gradients can be calculated during backpropagation, providing the

mathematical foundation necessary for the network to learn from its errors and adjust its parameters accordingly. The range of an activation function further defines its behavior by establishing the boundaries within which neuron outputs will fall, influencing how information is propagated through the network. Finally, monotonicity contributes to the learning process by helping maintain a consistent relationship between inputs and outputs, which in turn supports more stable and reliable convergence during training as the network progressively refines its understanding of the data.

1.9.2. Common Activation Functions with Examples

1. Sigmoid (Logistic Function)

The sigmoid activation function, also known as the logistic function, transforms inputs through the mathematical formula σ(x) = 1 divided by the quantity one plus e raised to the negative x, producing outputs that fall within the bounded range of zero to one. This function exhibits a distinctive smooth S-shaped curve that elegantly maps any real-valued input into a probability-like output, which has made it particularly valuable in scenarios where outputs need to be interpreted as probabilities, such as in the final layer of binary classification networks. Despite these useful characteristics, the sigmoid function carries a significant drawback known as the vanishing gradient problem, where for very high or very low input values, the function's gradient approaches zero, severely slowing down learning by effectively preventing weight updates in deeper layers of the network during backpropagation.

Example Use Case: Binary classification output layer

Python code

If a neuron receives input z = 2.0

z = 2.0

`sigmoid_output = 1 / (1 + math.exp(-2.0)) ≈ 0.88`

Interpretation: 88% probability of belonging to class 1

Derivative: σ'(x) = σ(x) * (1 - σ(x))

2. Tanh (Hyperbolic Tangent)

Formula: tanh(x) = (e^x - e^(-x)) / (e^x + e^(-x))

The hyperbolic tangent activation function, commonly referred to as tanh producing outputs that span the range from negative one to positive one. This function represents a significant improvement over the sigmoid in one key aspect, as its outputs are zero-centered, meaning values are symmetrically distributed around zero, which generally facilitates more efficient training by ensuring that gradients during backpropagation are not constrained to be all positive or all negative. Despite this advantage in centering, the tanh function remains susceptible to the vanishing gradient problem that plagues sigmoid-based networks, as its gradients similarly become extremely small for inputs that fall within the saturation regions at either end of its range, which can impede learning in deeper network architectures.

Example Use Case: Hidden layers where zero-centered outputs are beneficial

Python Code

If a neuron receives input z = 1.0

z = 1.0

`tanh_output = math.tanh(1.0) ≈ 0.76`

Output is centered around 0

Derivative: tanh'(x) = 1 - tanh²(x)

3. ReLU (Rectified Linear Unit)

Formula: ReLU(x) = max(0, x)

The Rectified Linear Unit, commonly known as ReLU produces outputs that range from zero to positive infinity. This simplicity yields exceptional computational efficiency, as the function requires only a threshold comparison rather than expensive exponential calculations, making it particularly attractive for training large-scale neural networks. Furthermore, ReLU effectively addresses the vanishing gradient problem for positive input values, since its gradient remains constant at one in this region, allowing gradients to flow freely through active neurons during backpropagation without diminishing. However, this activation function introduces its own significant limitation known as the dying ReLU problem, where neurons can become trapped in a state that outputs zero for all inputs, effectively rendering them permanently inactive and unable to recover or contribute further to the network's learning process.

Example Use Case: Most common choice for hidden layers

Python Code

Different inputs:

ReLU(3.5) = 3.5 # Positive input passes through

ReLU(-2.0) = 0.0 # Negative input becomes zero

ReLU(0) = 0.0 # Zero remains zero

Derivative:

ReLU'(x) = 1 if x > 0

`0 if x ≤ 0`

4. Leaky ReLU

Formula: LeakyReLU(x) = max(αx, x) where α is a small constant (e.g., 0.01)

Leaky ReLU addresses the fundamental limitation of the standard ReLU function by introducing a modification to handle negative input values, resulting in an output range that spans from negative to positive infinity. This activation function maintains the computational efficiency of its predecessor while solving the dying ReLU problem through a simple but effective adjustment, as it allows a small, non-zero gradient to persist even for negative inputs instead of setting them completely to zero. By permitting this small gradient flow for all values, Leaky ReLU ensures that neurons can

continue learning and updating their parameters regardless of whether they receive positive or negative input signals, preventing the permanent inactivation that plagues standard ReLU units and promoting more robust training across the entire network.

Example Use Case: Alternative to ReLU in deep networks

Python Code

With α = 0.01

LeakyReLU(3.5) = 3.5 # Positive input unchanged

LeakyReLU(-2.0) = 0.01 × -2.0 = -0.02 # Negative input has small value

Derivative:

text

LeakyReLU'(x) = 1 if x > 0

α if x ≤ 0

5. Softmax

Formula: softmax(x_i) = e^(x_i) / Σ(e^(x_j)) for all j

The softmax activation function transforms a vector of raw numerical scores into a probability distribution, producing outputs that each fall within the range of zero to one while collectively summing precisely to one. This specialized function accomplishes this by exponentiating each input value and then normalizing these exponentials by their sum across all outputs, effectively converting relative differences between scores into meaningful probabilities. Because of this property, softmax serves as the standard activation function for the output layer in multi-class classification problems, where it enables the network's final predictions to be interpreted as the probability that a given input belongs to each possible class, with the highest probability typically indicating the predicted category.

Example Use Case: Output layer for multi-class classification

Python Code

Three output neurons with raw scores [2.0, 1.0, 0.1]

scores = [2.0, 1.0, 0.1]

exp_scores = [math.exp(2.0), math.exp(1.0), math.exp(0.1)] ≈ [7.39, 2.72, 1.11]

sum_exp = 7.39 + 2.72 + 1.11 = 11.22

softmax_output = [7.39/11.22, 2.72/11.22, 1.11/11.22] ≈ [0.66, 0.24, 0.10]

Interpretation: 66% probability for class 1, 24% for class 2, 10% for class 3

1.9.3 When to Use Which Activation Function

Scenario	Recommended Activation	Reason

Hidden Layers	ReLU/Leaky ReLU	Fast computation, avoids vanishing gradients
Binary Classification Output	Sigmoid	Natural probability interpretation
Multi-class Classification Output	Softmax	Probability distribution over classes
Regression Output	Linear/None	Unbounded output for real values
RNNs/LSTMs	Tanh/Sigmoid	Historical use, bounded outputs

1.9.4. Implement a neural network to predict housing prices using Python

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.datasets import fetch_california_housing
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# Set random seeds for reproducibility
np.random.seed(42)
tf.random.set_seed(42)

class HousingPricePredictor:
    def __init__(self):
        self.model = None
```

```
    self.scaler_X = StandardScaler()
    self.scaler_y = StandardScaler()
    self.history = None

def load_data(self):
    """Load California housing dataset"""
    housing = fetch_california_housing()
    X = housing.data
    y = housing.target.reshape(-1, 1)  # Reshape for scaling
    feature_names = housing.feature_names

    print(f"Dataset shape: {X.shape}")
    print(f"Feature names: {feature_names}")
    print(f"Target range: ${y.min():.2f} - ${y.max()*1000:.2f}")

    return X, y, feature_names

def preprocess_data(self, X, y):
    """Preprocess and split the data"""
    # Split the data
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42
    )

    # Scale features
    X_train_scaled = self.scaler_X.fit_transform(X_train)
    X_test_scaled = self.scaler_X.transform(X_test)

    # Scale target (prices)
    y_train_scaled = self.scaler_y.fit_transform(y_train)
    y_test_scaled = self.scaler_y.transform(y_test)

   return X_train_scaled, X_test_scaled, y_train_scaled, y_test_scaled, y_train, y_test

def build_model(self, input_dim):
    """Build the neural network model"""
    model = keras.Sequential([
        layers.Dense(128, activation='relu', input_shape=(input_dim,)),
```

```
        layers.Dropout(0.2),
        layers.Dense(64, activation='relu'),
        layers.Dropout(0.2),
        layers.Dense(32, activation='relu'),
        layers.Dense(1)  # Output layer for regression (no activation)
    ])

    # Compile the model
    model.compile(
        optimizer='adam',
        loss='mse',  # Mean Squared Error for regression
        metrics=['mae']  # Mean Absolute Error
    )

    return model

def train_model(self, X_train, y_train, X_val=None, y_val=None, epochs=100):
    """Train the neural network"""
    # Create validation split if not provided
    if X_val is None:
        X_train, X_val, y_train, y_val = train_test_split(
            X_train, y_train, test_size=0.2, random_state=42
        )

    # Callbacks for better training
    callbacks = [
        keras.callbacks.EarlyStopping(
            patience=15,
            restore_best_weights=True,
            monitor='val_loss'
        ),
        keras.callbacks.ReduceLROnPlateau(
            factor=0.5,
            patience=10,
            min_lr=1e-7
        )
    ]
```

```
    # Train the model
    self.history = self.model.fit(
        X_train, y_train,
        validation_data=(X_val, y_val),
        epochs=epochs,
        batch_size=32,
        callbacks=callbacks,
        verbose=1
    )

def evaluate_model(self, X_test, y_test, y_test_original):
    """Evaluate the model performance"""
    # Make predictions
    y_pred_scaled = self.model.predict(X_test)

    # Inverse transform predictions to original scale
    y_pred = self.scaler_y.inverse_transform(y_pred_scaled)

    # Calculate metrics
    mse = mean_squared_error(y_test_original, y_pred)
    mae = mean_absolute_error(y_test_original, y_pred)
    r2 = r2_score(y_test_original, y_pred)

    # Convert to thousands of dollars for better interpretation
    mse_k = mse * 1000
    mae_k = mae * 1000

    print(f"\n{'='*50}")
    print("MODEL EVALUATION")
    print(f"{'='*50}")
    print(f"Mean Squared Error: ${mse_k:,.2f}")
    print(f"Mean Absolute Error: ${mae_k:,.2f}")
    print(f"R² Score: {r2:.4f}")
    print(f"{'='*50}")

    return y_pred, {'mse': mse, 'mae': mae, 'r2': r2}

def plot_training_history(self):
```

```
        """Plot training history"""
        if self.history is None:
            print("No training history available.")
            return

        fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5))

        # Plot loss
        ax1.plot(self.history.history['loss'], label='Training Loss')
        ax1.plot(self.history.history['val_loss'], label='Validation Loss')
        ax1.set_title('Model Loss')
        ax1.set_xlabel('Epoch')
        ax1.set_ylabel('Loss (MSE)')
        ax1.legend()
        ax1.grid(True)

        # Plot MAE
        ax2.plot(self.history.history['mae'], label='Training MAE')
        ax2.plot(self.history.history['val_mae'], label='Validation MAE')
        ax2.set_title('Model MAE')
        ax2.set_xlabel('Epoch')
        ax2.set_ylabel('MAE')
        ax2.legend()
        ax2.grid(True)

        plt.tight_layout()
        plt.show()

    def plot_predictions(self, y_test, y_pred):
        """Plot actual vs predicted prices"""
        plt.figure(figsize=(10, 6))

        # Convert to thousands of dollars for better visualization
        y_test_k = y_test.flatten() * 100
        y_pred_k = y_pred.flatten() * 100

        # Create scatter plot
        plt.scatter(y_test_k, y_pred_k, alpha=0.6)
```

```
        # Perfect prediction line
        max_val = max(y_test_k.max(), y_pred_k.max())
        min_val = min(y_test_k.min(), y_pred_k.min())
        plt.plot([min_val, max_val], [min_val, max_val], 'r--', lw=2)

        plt.xlabel('Actual Prices ($000)')
        plt.ylabel('Predicted Prices ($000)')
        plt.title('Actual vs Predicted Housing Prices')
        plt.grid(True, alpha=0.3)

        # Add R² to plot
        r2 = r2_score(y_test, y_pred)
        plt.text(0.05, 0.95, f'R² = {r2:.4f}', transform=plt.gca().transAxes,
                 bbox=dict(boxstyle="round,pad=0.3", facecolor="white", alpha=0.8))

        plt.tight_layout()
        plt.show()

    def run_complete_pipeline(self, epochs=100):
        """Run the complete pipeline"""
        print("Loading data...")
        X, y, feature_names = self.load_data()

        print("Preprocessing data...")
        X_train, X_test, y_train_scaled, y_test_scaled, y_train_orig, y_test_orig = self.pre-
process_data(X, y)

        print("Building model...")
        self.model = self.build_model(X_train.shape[1])

        print("Model architecture:")
        self.model.summary()

        print("\nTraining model...")
        self.train_model(X_train, y_train_scaled, epochs=epochs)

        print("\nEvaluating model...")
```

```
        y_pred, metrics = self.evaluate_model(X_test, y_test_scaled, y_test_orig)

        # Plot results
        self.plot_training_history()
        self.plot_predictions(y_test_orig, y_pred)

        return metrics

# Run the complete pipeline
if __name__ == "__main__":
    predictor = HousingPricePredictor()
    metrics = predictor.run_complete_pipeline(epochs=100)
```

Output:

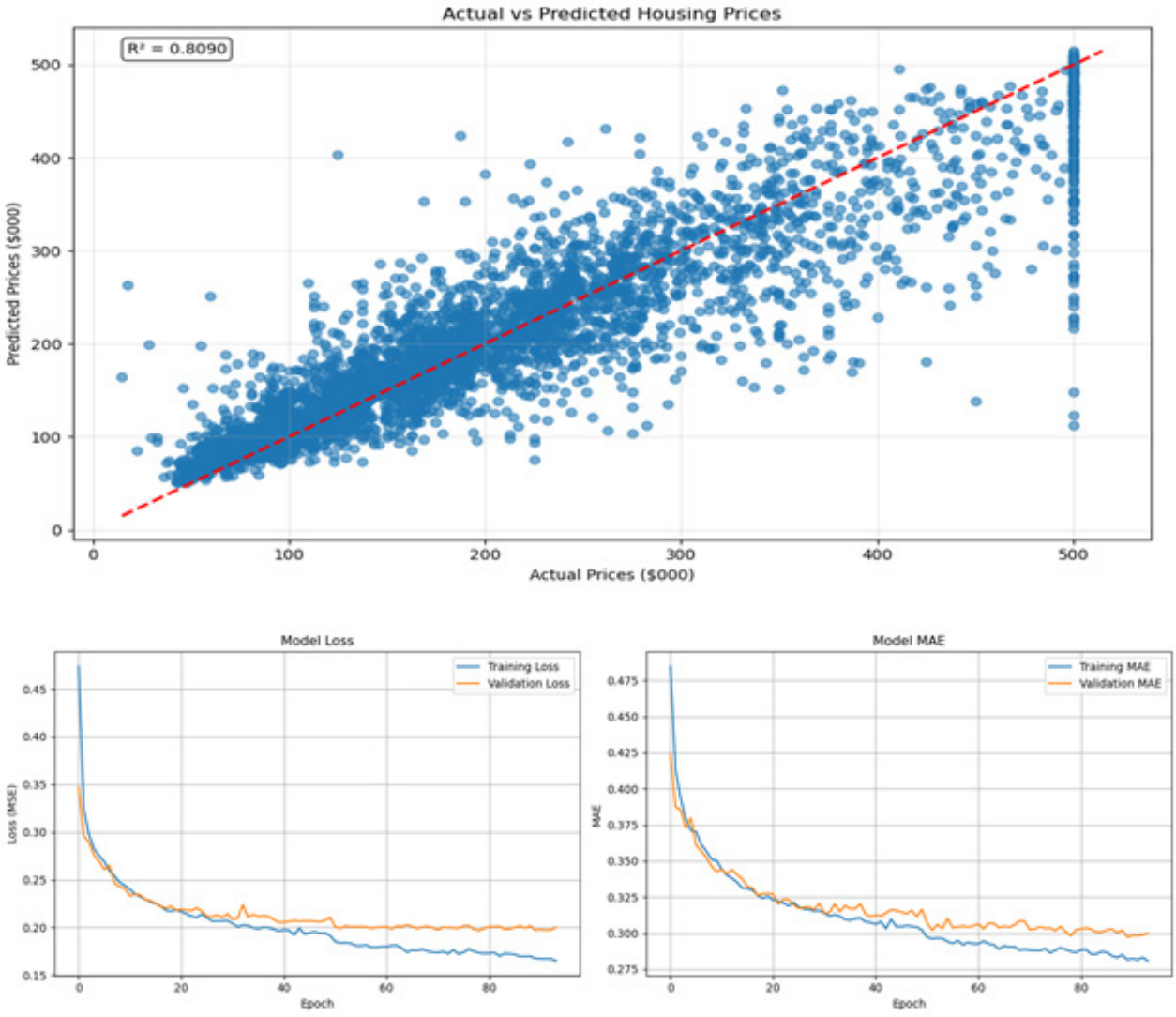


Figure 1.2 Actual Vs Predicted Housing Prices, Model Loss and Model MAE

Topic of Class Discussion

The Biological Brain vs. The Artificial Neural Network: A Useful Metaphor or a Misleading Oversimplification?

Chapter Summary

An Artificial Neural Network is a computing system inspired by the biological brain, designed to learn patterns and relationships from data. Its fundamental building block is the artificial neuron, which are interconnected in structured layers including input, hidden, and output layers. The true power of a neural network emerges from its ability to learn through a process of iterative guesswork and self-correction. Data flows forward through the network in a process called forward propagation to make a prediction. The network's performance is then evaluated by comparing its prediction to the known correct answer using a loss function. The resulting error is sent backward through the network via backpropagation, which identifies how much each connection contributed to the mistake. An optimization algorithm like Gradient Descent then uses this information to carefully adjust the weights and biases of the connections. A single perceptron is a simple model of a neuron but is limited to solving only linearly separable problems. The Multilayer Perceptron overcomes this limitation by connecting multiple layers of neurons, enabling it to learn complex, non-linear relationships. Activation functions like ReLU are crucial as they introduce non-linearity, allowing the network to model real-world data. Different activation functions are chosen for specific tasks, such as Softmax for multi-class classification output. Through this hierarchical feature learning, each successive layer in a deep network can recognize more abstract and complex patterns. The entire training cycle is repeated millions of times, allowing the network to gradually refine its internal model from the data. This process transforms the network from a collection of random connections into a powerful predictive model. Ultimately, ANNs serve as versatile function approximators capable of tasks ranging from image recognition to financial forecasting.

Exercises

A. Fill in the Blanks

1. The fundamental building block of an artificial neural network is the artificial _____.
2. The process of sending error information backward through the network to adjust weights is called _____.
3. The strength of the connections between neurons in an ANN is represented by _____.
4. The _____ layer of a neural network is responsible for delivering the final prediction or decision.

5. A single perceptron can only act as a _____ classifier, meaning it can only separate data with a straight line.
6. The _____ function is commonly used in the output layer for multi-class classification problems to output probabilities.
7. The core algorithm used to train Multilayer Perceptrons is _____ combined with Gradient Descent.
8. The _____ activation function, defined as max(0, x), is the default choice for hidden layers in modern networks.
9. The learning process of an ANN involves an iterative cycle of forward pass, error calculation, backpropagation, and _____ update.
10. The _____ Theorem states that an MLP with even one hidden layer can approximate any continuous function.

B. Multiple Choice Questions

1. What is the primary purpose of the hidden layers in an ANN?
   a) To receive raw input data
   b) To deliver the final output
   c) To perform complex computations and identify patterns
   d) To visualize the network structure
2. Which component of a neuron allows it to fine-tune its trigger point by adding an adjustment factor?
   a) Weight
   b) Activation Function
   c) Bias
   d) Input
3. The problem where gradients become extremely small, slowing or stopping training in deep networks, is known as:
   a) The Dying ReLU Problem
   b) The Exploding Gradient Problem
   c) The Vanishing Gradient Problem
   d) The Overfitting Problem
4. A Multilayer Perceptron (MLP) is more powerful than a single perceptron because it:
   a) Uses a simpler activation function
   b) Can learn complex, non-linear relationships
   c) Has only one layer of neurons
   d) Does not require a learning rate
5. In the context of backpropagation, what does the “chain rule” from calculus allow us to do?
   a) Calculate the forward pass more efficiently
   b) Determine the exact contribution of each weight to the final error

c) Choose the best activation function
d) Initialize the weights randomly

6. Which activation function is most appropriate for the output layer of a binary classification model?
a) ReLU
b) Softmax
c) Sigmoid
d) Linear

7. What is the key limitation of a single perceptron?
a) It cannot learn at all.
b) It can only solve linearly separable problems.
c) It is too computationally expensive.
d) It requires categorical data.

8. During training, the difference between the network's prediction and the correct answer is calculated by the:
a) Optimizer
b) Activation Function
c) Loss Function
d) Weight

9. Which of the following best describes the role of the "learning rate" during weight updates?
a) It defines the number of training epochs.
b) It controls the size of the steps taken during Gradient Descent.
c) It determines the number of hidden layers.
d) It is the final output of the model.

10. In the biological analogy for a perceptron, what do the "weights" represent?
a) The speed of the electrical signal
b) The strength or importance of the input signals
c) The nucleus of the cell
d) The length of the axon

A. Short Answer Questions

1. Briefly state the primary function of the input, hidden, and output layers in an ANN.
2. What is the main difference between a biological neuron and an artificial neuron in terms of their processing mechanism?
3. Why are non-linear activation functions essential in neural networks?
4. Explain the "Dying ReLU" problem in one or two sentences.
5. What is the purpose of the bias term in a perceptron?
6. In the context of the MLP example for handwritten digits, what kind of features might the first hidden layer learn to detect?

7. What is the role of an optimizer (like Gradient Descent) in the training process?
8. Why is Softmax used in the output layer for a multi-class classification task?
9. Name one key advantage that Leaky ReLU has over the standard ReLU activation function.
10. What does the Universal Approximation Theorem tell us about Multilayer Perceptrons?

B. Long Answer Questions

1. Describe the complete cycle of learning in an Artificial Neural Network. Your answer should detail the four key stages (forward pass, evaluation, backpropagation, and update) and explain the role of each component (weights, bias, activation function, loss function, optimizer) in this process.
2. Compare and contrast a single perceptron with a Multilayer Perceptron (MLP). Your answer should cover their structural differences, learning capabilities, the types of problems they can solve, and the key technical advancements (e.g., activation functions, learning algorithms) that make MLPs more powerful.
3. Explain the concept of backpropagation with a detailed example. Walk through the steps of a single forward and backward pass for a simple network, explaining how the error is calculated and propagated back to adjust the weights. You may use the example from section 1.8 of the document as a reference.
4. Discuss the importance of activation functions in neural networks. Choose three common activation functions (e.g., Sigmoid, Tanh, ReLU), describe their mathematical properties (range, shape), their derivatives, and explain scenarios where each would be appropriately used. Also, mention one limitation for each.
5. Outline the steps you would take to design and implement a neural network for a real-world regression problem, such as predicting housing prices. Your answer should cover data acquisition and preprocessing, model architecture design choices (number of layers, neurons, activation functions), the training process (including callbacks), and model evaluation. Refer to the Python code in section 1.9.4 to support your explanation.